

Real-Time Lens Distortion Correction: 3D Video Graphics Cards Are Good for More than Games

Michael R. Bax

Abstract. Optical lens systems suffer from non-linear radial distortion. Image-based applications require distortion compensation for the accurate location, registration and measurement of image features, and a real-time capability is desirable for interactive systems. A texture-mapping graphics accelerator can use corresponding meshes of distorted and undistorted nodes along with the dynamic distorted image to render high-quality distortion-corrected images at video framerates. Mesh generation, an error analysis, and performance results are presented. The proposed polar-based method is shown to have both more accuracy than a conventional grid-based approach and greater speed than the traditional method of using the CPU to transform each pixel individually.

Index Terms: lens distortion, correction, compensation, real-time, texture mapping.

I. Introduction

THE optical lens systems used in imaging equipment ranging from film cameras to endoscopes suffer from distortion artefacts, which detract from the quality of the images produced (see Fig. 1). In applications such as photogrammetry, computer vision, and medical imaging, the determination of and compensation for distortion is required to enable accurate location, measurement and registration of features in images [1]–[5].

While distortion correction may be applied offline in many cases, a real-time capability is desirable for systems that must interact with the environment or with a user in real time. It is possible to implement radial lens distortion correction at video framerates on current workstation central processing units (CPU's) [6], but correcting full-resolution colour video images in real time can require more processor power than is available.

In cases where the raw CPU power is not exceeded, the processor may nevertheless be concurrently required for other important real-time tasks such as navigation or tracking, making it impractical to spare much CPU time; distortion correction should ideally be performed with minimal impact on the CPU load.

II. Lens Distortion

Infinite series are needed to fully model non-linear lens distortion [1], [2], but in practice it is normally sufficient to model only the dominant radial distortion (also known as barrel or pincushion distortion) using a single parameter, κ_1 [3]. This is the model used here; it is assumed that the value of κ_1 is known.

The radial lens distortion is modelled as

$$r_u = r_d(1 + \kappa_1 r_d^2), \quad (1)$$

where r_u is the correct, undistorted radial distance to a point from the optical centre of the image, and r_d is the distorted radius. This relationship is illustrated in Fig. 2.

Given r_u , calculation of r_d requires the solution of this cubic equation. Applying Viète's substitution $r_d = w - \frac{1}{3\kappa_1 w}$ leads to



(a) Distorted image



(b) Undistorted image

Fig. 1. Radial lens distortion: before and after.

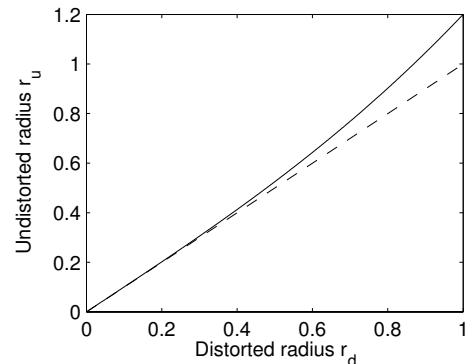


Fig. 2. The radial lens distortion model.

the quadratic form

$$(w^3)^2 - \frac{r_u}{\kappa_1} w^3 - \frac{1}{27\kappa_1^3} = 0 \quad (2)$$

and its solution

$$w = \sqrt[3]{\frac{r_u}{2\kappa_1} \pm \sqrt{\frac{r_u^2}{4\kappa_1^2} + \frac{1}{27\kappa_1^3}}}. \quad (3)$$

The value of r_d is found by substituting either of these roots for w .

III. Distortion Correction

A. Individual pixel resampling

The direct approach to reversing lens distortion is to resample the image using the inverse mapping. Simply choosing the closest integral pixel location for each point (nearest neighbour) results in poor visual quality, but bilinear interpolation between the four adjacent pixel values gives a satisfactory result at a reasonable additional computational cost.

Let $f(u)$ and $g(u)$ denote the value of the pixel at the location u in the input and output images respectively, and let u_0 be the optical image centre. The sampling position in the input image is therefore

$$u_f = u_0 + \frac{u_g - u_0}{1 + \kappa_1 r_d^2}. \quad (4)$$

Let $\Delta = (\Delta_1, \Delta_2)$ be the fractional part of the sampling position, and let $u_{f_{xy}} = u_{f_{00}} + (x, y)$ denote an adjacent pixel location. Bilinear interpolation gives

$$\begin{aligned} \alpha &= u_{f_{00}} + \Delta_2 (u_{f_{01}} - u_{f_{00}}) \\ \beta &= u_{f_{10}} + \Delta_2 (u_{f_{11}} - u_{f_{10}}) \\ g(u_g) &= \alpha + \Delta_1 (\beta - \alpha). \end{aligned} \quad (5)$$

Pixel-by-pixel resampling at video framerates is accurate but CPU-intensive. Even a fast processor has little time for other tasks, and if the image size and/or colour-depth are too large it will fail to keep up. Faster approaches such as simple linear compensation using image scaling [6] suffer from significant residual error. Dedicated, special-purpose hardware solves this problem [8], [9] but is typically inflexible and relatively expensive.

B. Local affine transformation with texture mapping

Inexpensive yet powerful PC 3D video accelerators, or graphics processing units (GPU's), have given the workstations the capability of rendering hundreds of millions of texture-mapped pixels per second; GPU-based image-processing algorithms can free the CPU for other tasks.

The GPU accepts one or more images (known as textures), a list of 3D triangle vertex co-ordinates $x = (x, y, z)$, and a corresponding list of 2D texture co-ordinates $u = (u, v)$. The rendering process computes the displayed position of each vertex, then interpolates between the corresponding texture co-ordinates to determine the texture location to sample in order to fill the pixels in each triangle.

The goal of texture-mapping in a GPU is to warp images fast—but it can also be used to *unwarp* distorted images!

1) *Cartesian-based vertex mapping*: The straightforward approach to exploiting the GPU is to tessellate the distorted input image into a set of triangles in a uniformly-spaced grid [10], [11], as shown in Fig. 3(a).

The original grid intersection positions become the list of texture co-ordinates u_i , and the distortion-corrected positions form a list of triangle vertex positions x_i for rendering. Following the one-time calculation of these lists, a stream of distorted images is fed to the GPU as a sequence of replacement textures.

The trade-off here is accuracy for speed. The vertices are correctly located, but the interior of each triangle undergoes an affine transformation. As a result, the interpolated sampling positions introduce distortion-correction errors in the *positions* of input image content within the corrected output image.

The worst-case error may however be reduced by orienting the triangles in each quadrant to minimise their lengths in the radial direction, as shown in Fig. 3(b).

2) *Polar-based vertex mapping*: The lens distortion model is parametrically polar. Tessellating the distorted image on a polar basis is a natural fit, facilitating analysis and permitting straightforward optimisation of the vertex locations: this yields a more efficient use of triangles and avoids localised peaks in the error field. It also allows the triangles to be more equilateral in shape, which for a given triangle area reduces the affine transformation error.

The approach proposed here is to divide the input image into concentric annuli and apply a radially affine transform to approximate the lens distortion correction function. In order to tessellate the annuli into triangles for rendering, the perimeters of the rings are broken up into piecewise-linear segments.

If the number of segments (and hence triangles) is constant for each perimeter, the maximum dimension of the triangles grows in size as the radial distance increases. In order to offset this, additional triangles must be introduced. In this method the additional triangles are introduced along 6 equispaced radius vectors as shown in Fig. 3(c), reducing the deformation from equilateral form of the propagated triangles.

As a result, the i th complete annulus from the image centre adds $6i$ vertices and $6(2i-1)$ triangles; the first n complete annuli contain $3n(n+1) + 1$ vertices describing $6n^2$ triangles. Note however that these numbers apply only to circular images such as those from an endoscope. In general the borders of a rectangular image will clip much of the outer annuli, and triangles falling outside the borders of the output image need not be rendered.

The remaining element is the determination of the width of a given annulus. With a Cartesian-based vertex approach it is simple to determine the number of triangles; the polar-based case on the other hand lends itself more directly to minimising the number of triangles subject to a given goal of maximum distortion-correction error. This is achieved if each annulus has the greatest width that does not cause the intra-annulus error to exceed the specified limit.

3) *Analysis*: In order to maximise the width of a given annulus without causing its internal distortion-correction error to exceed the desired threshold when undergoing affine transformation, the maximum error within its boundaries must be known.

Consider the i th annulus, with its interior perimeter at the radius r_{d_i} and of radial width w_{d_i} in the lens-distorted input

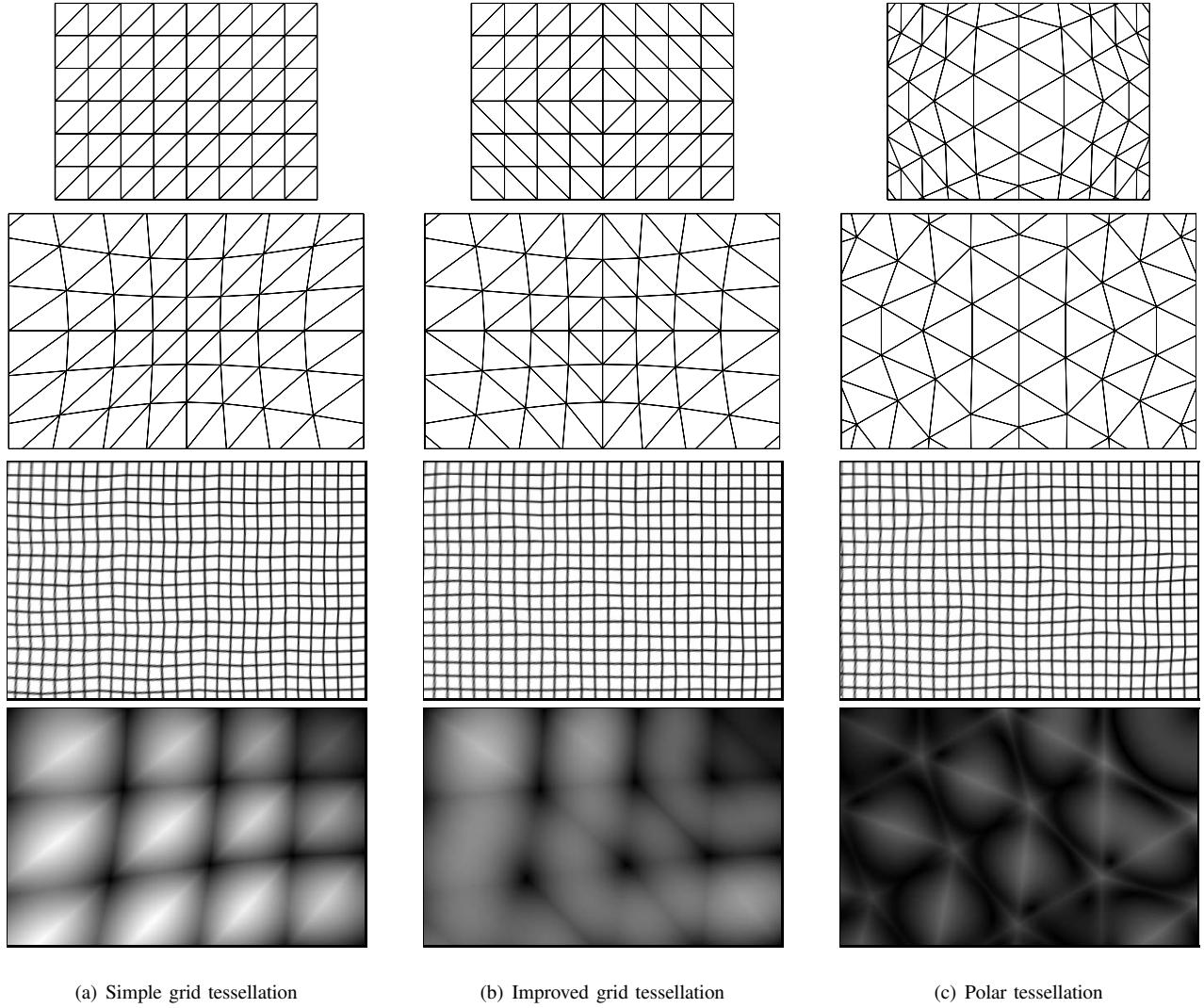


Fig. 3. From top to bottom: input image texture co-ordinates; output image vertex co-ordinates; lower-left quadrant of a distortion-corrected output image; lower-left quadrant of the output image pixel error (black is zero). The input image was divided into a similar number of triangles in each case.

image. A point within this annulus has radius

$$r_d(\alpha) = r_{d_i} + \alpha w_{d_i} \quad (6)$$

for some $\alpha \in [0, 1)$. The correct distortion-free radius for this point in the output image is therefore

$$r_u(\alpha) = (r_{d_i} + \alpha w_{d_i}) \left(1 + \kappa_1 (r_{d_i} + \alpha w_{d_i})^2 \right), \quad (7)$$

but affine transformation of this point results in the actual output image radius

$$\begin{aligned} \tilde{r}_u(\alpha) &= (1 - \alpha)r_{d_i} (1 + \kappa_1 r_{d_i}^2) \\ &\quad + \alpha (r_{d_i} + w_{d_i}) \left(1 + \kappa_1 (r_{d_i} + w_{d_i})^2 \right). \end{aligned} \quad (8)$$

The error is

$$\begin{aligned} e(\alpha) &= \tilde{r}_u(\alpha) - r_u(\alpha) \\ &= \kappa_1 w_{d_i}^2 \alpha (1 - \alpha) (\alpha w_{d_i} + 3r_{d_i} + w_{d_i}), \end{aligned} \quad (9)$$

which reaches its maximum where

$$\begin{aligned} \frac{d}{d\alpha} e(\alpha) &= \kappa_1 w_{d_i}^2 (3r_{d_i} + w_{d_i} - 6r_{d_i} \alpha - 3w_{d_i} \alpha^2) \\ &= 0, \end{aligned} \quad (10)$$

giving

$$\alpha_{e_{\max}} = \frac{2s - 6r_{d_i}}{6w_{d_i}}, \quad s = \sqrt{9r_{d_i}^2 + 9w_{d_i}r_{d_i} + 3w_{d_i}^2}. \quad (11)$$

Substituting $\alpha_{e_{\max}}$ into (9) gives

$$e_{\max} = \frac{(2s + 3r_{d_i})(s - 3r_{d_i})^2 \kappa_1}{27}. \quad (12)$$

This does not have a neat closed-form solution for w_{d_i} given r_{d_i} and $e_{\max}$, but it is straightforward to solve numerically.

Unfortunately, the annulus is not transformed monolithically; if it were simply scaled radially to match the lens-distorted inner and outer perimeters in the output image, the result in (12) would be the only error. The annulus is however tessellated into triangles for rendering, each of which undergoes individual affine transformation.

Non-radially aligned lines within the triangles are mapped to straight lines in the output image; exact distortion compensation would transform them to curves. This introduces another error component with approximately the same range as e_α . Both kinds

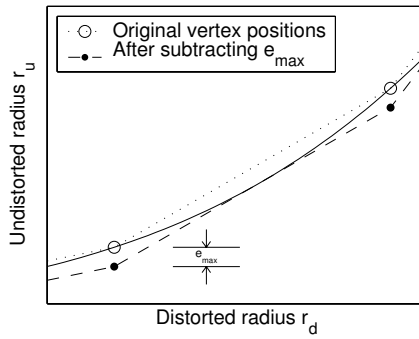


Fig. 4. Vertex position adjustment: the dotted line shows the initial affine approximation of the non-linear curve, and the dashed line shows the approximation after reducing the radii of the vertices by $e_{\max}$ to cancel tessellation error.

of error are centrifugal, increasing the radii of points in the output image beyond their corrected values.

These two sources of error have approximately the same range. The additional error introduced by triangular tessellation may therefore be counterbalanced by moving the previously-computed output image vertices a distance of $e_{\max}$ towards the image centre, as illustrated in Fig. 4. This in effect negates the range of e_{α} so that it tends to cancel the tessellation error component.

IV. Implementation

A C framework using the Microsoft DirectX 9.0 API was written to compare lens distortion correction using:

- individual pixel resampling by the CPU
- texture-mapped affine transformation by the GPU.

Both methods used bilinear interpolation. A look-up table was used to optimise CPU-based resampling. Ping-pong frame buffers in 24-bit packed RGB format were used to simulate an incoming colour video stream of 640×480 pixels (the full scanline resolution of an NTSC signal, reformatted for square pixels on a display with a 4:3 aspect-ratio). The display depth was set to 32-bit XRGB format, and a pair of texture maps were used to allow the display to use one while the other was updated with incoming video data.

The value of κ_1 was set to 0.2 (scaled for a normalised maximum inscribed radius of 1 in the input image). This value is representative of endoscope telescopes such as the 0° Karl Storz 50200 A.

The resolution of the output image was set to 866×574 . This is the largest rectangular region that is entirely filled when a 640×480 image is distortion-corrected using $\kappa_1 = 0.2$ with no loss of information (1:1 sampling of the centre of the input image and oversampling elsewhere).

Three increasingly demanding targets for $e_{\max}$ were chosen: a maximum desired error of 1, 0.1, and 0.01 pixels respectively. A triangle mesh was designed using the analysis in section III-B.3 and those triangles not visible in the output image were culled. The remaining triangles were totalled and Cartesian-based meshes were selected to match the number of output image triangles as closely as possible while constraining the row height to column width ratio in order to obtain approximately square cells. The grids were divided into triangles in two ways: using first the simpler approach with triangles of constant orientation,

and second the improved orientation of each diagonal to maximise its angle of intersection with the radius vector.

The pixel error at each output image pixel was determined by measuring the deviation of the actual sampling position from its correct ideal location in the input image. Real-time performance measurements were made on a 900 MHz Intel Pentium III computer with a 32 MB NVIDIA GeForce2 Go GPU and 512 MB of system RAM, running Microsoft Windows XP. Each test was run for 40 seconds; the data from the first 10 seconds was discarded to allow performance stabilisation. Elapsed times were recorded using a high-resolution hardware counter with sub-millisecond accuracy.

V. Results

As expected the error-free CPU-based method of individually resampling each pixel was too slow to keep up with the NTSC video rate of 30 frames per second when processing this amount of data (see Table I). The real-time correction of radial lens distortion in, and the subsequent display of, the equivalent of a 24-bit colour NTSC video stream demanded more power than was available from the CPU.

All the texture-mapping methods comfortably exceeded the video threshold of 30 frames per second. The optimum trade-off of pixel error against triangle count appears to be in the region of $e_{\max} = 0.1$; at the lower tolerance of $e_{\max} = 0.01$ pixels the framerate began to fall.

In each group of meshes with similar numbers of triangles, the Cartesian-based vertex mapping with improved triangle orientation lowered maximum pixel error in the input image by approximately 20% compared to a simple grid.

Polar-based vertex mapping performed substantially better than either of the other mapping methods, beating the simple and improved methods by about 50% and 40% respectively. The largest measured error was within 20% of the design $e_{\max}$ in each case, improving significantly for higher values of $e_{\max}$.

VI. Conclusion

If using a simple Cartesian-based tessellation, a useful improvement in accuracy can be easily achieved through a simple layout optimisation. If still greater accuracy is required, polar-based tessellation can more than double the improvement made by the Cartesian-based optimisation.

The GPU tested here is not a recent model, yet the texture-mapping methods described in this paper easily surpass the video threshold of 30 frames per second. This demonstrates a real-time performance capability on inexpensive, commodity hardware.

Newer GPU's feature programmable pixel shaders, which could be used to implement a real-time version of the per-pixel lens distortion compensation algorithm used on the CPU. Although there would be no pixel error with this approach, it is not expected to be as fast as conventional texture-mapping since it cannot be optimised and implemented directly at the hardware level in a fixed-function pipeline. In addition, pixel shader programming imposes an additional layer of complexity and the API's have yet to stabilise.

An efficient multi-threaded implementation of the pixel resampling algorithm running on a fast dual-CPU workstation can be

TABLE I

PERFORMANCE RESULTS FOR THE DISTORTION-CORRECTION METHODS DISCUSSED IN SECTION III. THE CARTESIAN-BASED METHOD IS LISTED FIRST WITHOUT, AND THEN WITH, QUADRANT-BASED HYPOTENUSE ORIENTATION; THE METHODS ARE GROUPED BY TRIANGLE COUNT.

Lens distortion correction method	Design $e_{\max}$ [pixels]	Number of triangles	Maximum error [pixels]	RMS error [pixels]	Time [ms]	Frames per second
Individual pixel resampling	—	—	0	0	168	6
Cartesian-based vertex mapping (simple)	—	258	1.888	0.879	26	39
Cartesian-based vertex mapping (improved)	—	256	1.495	0.729	26	39
Polar-based vertex mapping	1.00	254	0.902	0.351	26	39
Cartesian-based vertex mapping (simple)	—	2026	0.231	0.106	25	39
Cartesian-based vertex mapping (improved)	—	2020	0.188	0.088	25	39
Polar-based vertex mapping	0.10	2052	0.105	0.036	25	39
Cartesian-based vertex mapping (simple)	—	19494	0.024	0.011	28	36
Cartesian-based vertex mapping (improved)	—	19472	0.019	0.009	28	36
Polar-based vertex mapping	0.01	19452	0.012	0.004	28	36

expected to exceed the 30 frames-per-second threshold; this would however dominate the available CPU time. This is undesirable in environments where the CPU is simultaneously required for other tasks.

The bus between the display subsystem and the rest of the computer system is potentially a secondary bottleneck; since lens distortion generally compresses the periphery of an image, transmitting the smaller distorted image over this bus is more efficient. Although doing so translates to a 38% reduction in bandwidth for NTSC colour video images with $\kappa_1 = 0.2$, AGP bandwidth is an order of magnitude greater; this is only likely to be an issue for very large or highly-distorted images.

There are significant additional potential benefits of using the GPU texture-mapping technique if further manipulation of the distortion-corrected image is required. Once the input image has been uploaded into texture memory, the power of the GPU is available for additional rotation, scaling, translation, masking, blending and other operations with very little to no additional load. In applications such as augmented reality, virtual endoscopy, and other areas where the GPU either already is or can be used to project or composite images from multiple sources, lens distortion correction comes as a virtually resource-free bonus.

References

- [1] C. C. Slama, C. Theurer, and S. W. Henriksen, Eds., *Manual of Photogrammetry*, 4th ed. Amer. Soc. Photogrammetry and Remote Sensing, Jan. 1980.
- [2] J. Weng, P. Cohen, and M. Herniou, "Camera calibration with distortion models and accuracy evaluation," *IEEE Trans. Patt. Anal. Machine Intell.*, vol. 14, no. 10, pp. 965–980, Oct. 1992.
- [3] R. Y. Tsai, "A versatile camera calibration technique for high-accuracy 3D machine vision metrology using off-the-shelf TV cameras and lenses," *IEEE J. Robotics Automat.*, vol. 3, no. 4, pp. 323–344, Aug. 1987.
- [4] W. E. Smith, N. Vakil, and S. A. Maislin, "Correction of distortion in endoscope images," *IEEE Trans. Med. Imag.*, vol. 11, no. 1, pp. 117–122, Mar. 1992.
- [5] H. Haneishi, Y. Yagihashi, and Y. Miyake, "A new method for distortion correction of electronic endoscope images," *IEEE Trans. Med. Imag.*, vol. 14, no. 3, pp. 548–555, Sept. 1995.
- [6] R. Shahidi, M. R. Bax, C. R. Maurer, Jr., J. A. Johnson, E. P. Wilkinson, B. Wang, J. B. West, M. J. Citardi, K. H. Manwaring, and R. Khadem, "Implementation, calibration and accuracy testing of an image-enhanced endoscopy system," *IEEE Trans. Med. Imag.*, vol. 21, no. 12, pp. 1524–1535, Dec. 2002.
- [7] M. R. Bax, R. Khadem, J. A. Johnson, E. P. Wilkinson, and R. Shahidi, "Image-enhanced endoscopy calibration and image registration accuracy testing," in *Proc. SPIE Conf. Med. Imag.: Visualization, Image-Guided Procedures, and Display*, S. K. Mun, Ed., vol. 4681, May 2002, pp. 119–126.
- [8] F. M. Waltz, "Implementation of real-time perspective correction," in *Proc. SPIE Conf. Automated Inspection and High Speed Vision Architectures*, vol. 849, Cambridge, Massachusetts, USA, Nov. 1988, pp. 179–83.
- [9] A. G. J. Nijmeijer, M. A. Boer, C. H. Slump, M. M. Samson, M. J. Bentum, G. J. Laanstra, H. Snijders, J. Smit, and O. E. Herrmann, "Correction of lens-distortion for real-time image processing systems," in *Proc. IEEE Workshop on VLSI Signal Processing*, L. D. J. Eggermont, P. Dewilde, E. Deprettere, and J. Van Meerbergen, Eds., Veldhoven, the Netherlands, Oct. 1993, pp. 316–324.
- [10] D. A. Butler and P. K. Pierson, "A distortion-correction scheme for industrial machine-vision applications," *IEEE Trans. Robotics Automat.*, vol. 7, no. 4, pp. 546–551, Aug. 1991.
- [11] J. C. A. Fernandes, M. J. O. Ferreira, J. A. B. C. Neves, and C. A. C. Couto, "Fast correction of lens distortion for image applications," in *Proc. IEEE Int. Symp. Industrial Electronics*, vol. 2, July 1997, pp. 708–712.